

Kubernetes ConfigMaps and Secrets Made Simple

Storing configuration settings and passwords directly in your application code is a major security risk. This guide teaches you how to use Kubernetes ConfigMaps and Secrets to manage your application settings safely and easily.

What you'll learn:

- 1 Storing Public Settings with ConfigMaps
- 2 Keeping Secrets Safe and Hidden
- 3 Reading Your ConfigMap Inside a Pod
- 4 Decoding Secrets for Verification

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mrmanrajsingh

1

TIP 1 OF 4

Storing Public Settings with ConfigMaps

ConfigMaps are like environment files for your Kubernetes cluster. They store non-sensitive data like URLs, port numbers, or feature flags. Instead of hardcoding these values inside your application code, you define them in a ConfigMap. This allows you to change your application's behavior on the fly without rebuilding your container image.

```
$kubectl create configmap my-app-config --from-literal=API_URL="https://api.example.com" --from-literal=
```

Cloud & DevOps Daily

Kubernetes ConfigMaps & Secrets: Manage App Config Safely

Saturday, September 19, 2026

Swipe to tip 2 >> linkedin.com/in/mrmanrajsingh

2

TIP 2 OF 4

Keeping Secrets Safe and Hidden

Secrets are designed to store sensitive data like database passwords, API keys, and TLS certificates. Kubernetes automatically base64 encodes these values to prevent accidental exposure on your screen. You should use Secrets whenever you are handling data that shouldn't be visible to everyone on your team.

```
$kubectl create secret generic my-db-secret --from-literal=DB_PASSWORD="MySecurePassword123"
```

Cloud & DevOps Daily

Kubernetes ConfigMaps & Secrets: Manage App Config Safely

Saturday, September 19, 2026

Swipe to tip 3 >> linkedin.com/in/mrmanrajsingh

3

TIP 3 OF 4

Reading Your ConfigMap Inside a Pod

To make your application use these settings, you must inject the ConfigMap into your Pod. The easiest way to do this is by loading the ConfigMap as environment variables. When the container starts, it will read these variables just like any local environment variable.

```
$spec:
  containers:
  - name: my-app
    image: nginx
    envFrom:
    - configMapRef:
        name: my-app-config
```

Cloud & DevOps Daily

Kubernetes ConfigMaps & Secrets: Manage App Config Safely

Saturday, September 19, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Decoding Secrets for Verification

Kubernetes Secrets are stored in base64 encoding, which is not true encryption but protects them from quick glances. To verify the actual value stored inside a secret, you can retrieve the YAML output and run it through a base64 decoder. This is extremely helpful when debugging connection issues.

```
$kubectl get secret my-db-secret -o jsonpath="{.data.DB_PASSWORD}" | base64 --decode
```

Cloud & DevOps Daily

Kubernetes ConfigMaps & Secrets: Manage App Config Safely

Saturday, September 19, 2026

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

- + Use ConfigMaps for public settings like URLs and Secrets for private settings like API keys.
- + Keep your YAML files in a secure repository, but never commit actual raw passwords to Git.
- + Always decode secrets locally to verify their values instead of guessing during a deployment failure.
- + Use the `envFrom` field in your Pod spec to quickly load all ConfigMap keys as environment variables.
- + Name your ConfigMaps and Secrets clearly so your team knows exactly which microservice they belong to.

Watch Out For

- ! Confusing base64 encoding with real encryption; base64 is easily decoded by anyone.
- ! Updating a ConfigMap and expecting a running Pod to immediately apply the changes without a restart.
- ! Forgetting to create the ConfigMap or Secret before deploying the Pod, causing the Pod to get stuck in `CreateContainerConfigError`.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mrmanrajsingh